EAD

EDUCAÇÃO A DISTÂNCIA

UNIDADE II

Programação Orientada a Objetos I

Prof. Me. Tarcísio Peres

Conteúdo da Unidade II

- Sintaxe, tipos e declarações;
- Variáveis, operadores, *strings*, vetores;
- Object;
- Entrada e Saída;
    - CLR e BCL;
    - 3 camadas, MVC, MVVM.

Programação C# – Sintaxe

- Um programa típico é composto por declarações de namespace, classes e métodos.
- Embora não haja um limite específico para o número de caracteres em uma única linha, é prática comum manter linhas de código relativamente curtas para melhor legibilidade.
- A linguagem é fortemente tipada, ou seja, uma vez que uma variável é declarada para um tipo particular, seu tipo não pode ser alterado implicitamente.
- Parênteses () são frequentemente usados para envolver listas de parâmetros em definições de método e chamadas. As chaves { } delimitam blocos de código, como classes, métodos ou estruturas condicionais.
- Caracteres especiais, como ponto e vírgula (;), denotam o fim de uma instrução.
    - O # faz com que instruções sejam processadas antes da compilação do código, utilizado para incluir *namespaces* ou definir condicionais.
    - Há também um conjunto robusto de palavras-chave, como *class, int, void* e *if.*
    - Comentários são denotados por // para uma única linha, e /* */ para múltiplas linhas.

Programação C# – Sintaxe

- A maioria das convenções de codificação sugere usar nomes em inglês sem acento.

- Espaços não são permitidos em identificadores em C#. Por exemplo, uma variável nomeada int minha variável = 10 resultará em erro.

Convenções de nomenclatura comumente usadas:

Nome	Convenção	Exemplo
Classes e estruturas	PascalCase	PublicClass, MyStruct
Métodos	PascalCase (públicos e privados)	CalculateTotal(), PrintValues()
Propriedades	PascalCase	TotalAmount, LastItem
Eventos	PascalCase	ButtonClick, ValueChanged
Namespaces	PascalCase	System.Drawing, Microsoft.EntityFrameworkCore
Variáveis e parâmetros	camelCase	totalAmount, listItems
Membros de dados privados	camelCase do prefixo '_'	_privateValue, _initialCount
Interfaces	'I' seguido de PascalCase	IEnumerable, IDisposable
Constantes	PascalCase	MaxValue, DefaultSize
Campos readonly	'_' seguido e camelCase	_readonlyField

Fonte: autoria própria.

Programação C# – Namespace

- É um contêiner para um conjunto de identificadores que fornece uma maneira de agrupar funções, classes, estruturas e outros tipos sob um nome, evitando conflitos entre nomes e códigos maiores e mais complexos.
- São usados principalmente para organizar o código e proporcionar uma forma de evitar colisões de nomes entre diferentes bibliotecas ou partes do *software.*
- É importante diferenciar *namespaces* de *assemblies* no C#. O primeiro organiza o código logicamente, enquanto o segundo faz isso fisicamente. Um único *assembly* pode conter tipos de vários *namespace*. *Assembly* é uma unidade de implantação, como um arquivo DLL ou EXE.

```
namespace Criaturas.Mar
{
   public class Kraken
   {
// Implementação da classe Kraken
   }
}
```

Fonte: autoria própria.

Programação C# – Tipos numéricos

Podem ser categorizados em dois grupos: de valor integral e de ponto flutuante:

- tipos integrais representam números inteiros, ou seja, sem uma parte decimal.
- tipos de ponto flutuante representam números que têm uma parte decimal.

- Optar pelo tipo correto não apenas garante que os dados sejam representados corretamente, mas também otimiza o uso da memória.

Tipo integral	Descrição	Tipo flutuante	Descrição
byte	8 bits, 0-255	float	32 bits, precisão simples
sbyte	8 bits, -128-127	double	64 bits, precisão dupla
int	32 bits, signed	decimal	Alta precisão, fixo
uint	32 bits, unsigned		
shoty	16 bits, signed		
ushort	16 bits, unsigned		
long	64 bits, signed		
ulong	64 bits, unsigned		

Fonte: autoria própria.

Programação C# – Tipos numéricos

- Foi usada a letra F (*float*) após o número 88.7 (linha 6), e a letra M (decimal) após o número 1000000.75 (linha 10). Esses sufixos, em C#, são essenciais para diferenciar e especificar os tipos de dados numéricos literais, garantindo que sejam interpretados conforme o desejado.
- Também ajudam os programadores a evitar erros de compilação e comportamentos inesperados.
- O .NET Framework fornece classes numéricas que oferecem uma ampla gama de funcionalidades. São elas: System.Math, System.Numerics.BigInteger e System.Decimal.

```
1.  // Representando a idade de deuses gregos com tipos integrais.
2.  int idadeZeus = 3000; // Zeus viveu por um longo tempo, assim, o tipo
int é uma boa escolha.
3.  byte idadeEros = 20); // Eros é é frequentemente representado como um
jovem, por isso um byte é suficiente.
4.  // Comparando a força de dois deuses usando ponto flutuante.
5.  double forcaHercules = 99.5; // Hercules é quase perfeito em força.
6.  float forcaAthena = 88.7F; // Athena, sendo mais estratégica, tem uma
força menor.
7.  // Calculando o total de ouro em Dracmas no Monte Olimpo usando
decimal para alta precisão.
8.  decimal ouroOlimpo = 1000000.75M;
```

Fonte: autoria própria.

Programação C# – System.Math

- Fornece métodos estáticos para operações matemáticas básicas, bem como algumas constantes matemáticas úteis.

- Propriedades - E: retorna a base dos logaritmos naturais (aproximadamente 2,7180) e PI: retorna o valor de pi (aproximadamente 3,14159).

Alguns métodos:

- Abs(): retorna o valor absoluto de um número.
- Max(): retorna o maior de dois números.
- Round(): arredonda um valor para o inteiro mais próximo.
- Sqrt(): retorna a raiz quadrada de um número.

Programação C# – System.Numerics.BigInteger

- Representa um número inteiro de precisão arbitrária. Isso significa que ele pode ser usado para armazenar e operar em números inteiros muito grandes para serem representados pelos tipos numéricos padrão.

Alguns métodos:

- Abs(): retorna o valor absoluto.
- Add(): adiciona dois valores BigInteger.
- Divide(): divide um valor BigInteger por outro.
- Multiply(): multiplica dois valores BigInteger.
- Subtract(): subtrai um valor BigInteger de outro.

Programação C# – System.Decimal

- Embora decimal seja um tipo primitivo em C#, também tem métodos associados que podem ser usados para operações com precisão.

Alguns métodos:

- Add(): adiciona dois valores decimais.
- Ceiling(): retorna o menor número inteiro que é maior ou igual ao número decimal especificado.
- Divide(): divide um decimal por outro.
- Floor(): retorna o maior número inteiro menor ou igual ao número decimal especificado.
- Round(): arredonda um valor decimal para o número inteiro mais próximo.

Programação C# – Bool

- É fundamental para a programação e está enraizado em uma lógica binária (*true* ou *false*).
- Operadores como ==, !=, , <=, e >= comparam valores e retornam um resultado booleano.

```
1.  int troianos = 1000;
2.  int gregos = 900;
3.
4.  bool gregosSaoNumerosos = gregos> troianos; // Isso será avaliado como
false
```

- Operadores lógicos: && (E), || (OU) e ! (NÃO). Eles permitem combinar ou inverter valores booleanos.

```
1.  bool aresApoiaGregos = true;
2.  bool athenaApoiaGregos = false;
3.
4.  bool gregosTemApoioDivino = aresApoiaGregos || athenaApoiaGregos;//
Isso será avaliado como true
```

Fonte: autoria própria.

Programação C# – Boolean

- No .NET Framework existe uma estrutura chamada Boolean que representa um valor booleano (verdadeiro ou falso). É importante notar que bool é simplesmente um *alias* (uma espécie de apelido) para System.Boolean.
- Propriedades: TrueString retorna a string “True”; e FalseString, a string “False”.

Métodos:

- ToString(): converte o valor booleano do objeto atual em sua representação de *string* equivalente (“True” ou “False”).
- Equals(Object obj): retorna um valor que indica se uma instância é igual a um objeto especificado.
- GetType(): obtém o Type da instância atual.
- Parse(String value): converte a representação de *string* especificada de um valor lógico em seu equivalente Boolean.
- TryParse(String value, out Boolean result): tenta converter a representação de string especificada de um valor lógico em seu equivalente Boolean, retornando um valor que indica se a conversão foi bem-sucedida.

Programação C# – Cadeia de caracteres

- Uma cadeia de caracteres – mais comumente chamada de *string* em linguagens de programação – representa uma sequência de caracteres que pode ser usada para representar texto.
- É um objeto, ao contrário de algumas outras linguagens que a tratam como um tipo primitivo. Por ser um objeto, temos à disposição uma série de métodos e propriedades úteis para manipulação.
- Um aspecto fundamental das *strings* em C# é a imutabilidade, ou seja, uma vez criada uma *string*, ela não pode ser alterada.
    - Qualquer operação que pareça modificar uma *string* está na verdade criando uma nova *string.*
    - A linguagem C# também oferece a facilidade de interpolação de *string*, utilizando-se o cifrão ($) antes de iniciar uma *string* e { } para delimitar as expressões ou variáveis que desejamos inserir nela, permitindo que os valores sejam inseridos diretamente dentro da *string*.

Programação C# – Cadeia de caracteres

- *strings*

```
1.  string descricao = "Hércules luta contra a hidra de Lerna.";
2.  bool temHidra = descricao.Contains ("hidra");
3.  Console.WriteLine(temHidra); // Resultado: true
4.
5.  string mensagemAthena = "Defenda Atenas!;
6.  string mensagemApolo = "E ilumine o caminho.";
7.  string mensagemCombinada = mensagemAthena + ""+ mensagemApolo;
8.  Console.WriteLine(mensagemCombinada); // Resultado: Defenda Atenas! E
ilumine o caminho.
9.
10. rint numeroDeTridentes = 3;
11. string declaracao = $"Poseidon tem {numeroDeTridentes} tridentes.";
12. Console.WriteLine(declaracao); // Resultado: Poseidon: Poseidon tem 3
tridentes.
```

Interatividade

Qual das seguintes opções corretamente descreve um uso válido de tipos numéricos, booleanos e strings em C#, além de demonstrar a sintaxe correta?

a) Int number = "100"; bool isPositive = number > 0; string message = isPositive.ToString();

b) Double rate = 0.5; bool isActive = (rate = 1); string status = isActive ? "Ativo" : "Inativo";

c) String name = "CSharp"; bool isValid = name.Length > 5; int length = isValid;

d) Bool isEven = true; int num = 10; string result = isEven == (num % 2 == 0) ? "Par" : "Ímpar";

e) Long bigNumber = 9223372036854775807; bool isLarge = bigNumber > int.MaxValue; string description = "Número grande: " + isLarge;

Resposta

Qual das seguintes opções corretamente descreve um uso válido de tipos numéricos, booleanos e strings em C#, além de demonstrar a sintaxe correta?

a) Int number = "100"; bool isPositive = number > 0; string message = isPositive.ToString();

b) Double rate = 0.5; bool isActive = (rate = 1); string status = isActive ? "Ativo" : "Inativo";

c) String name = "CSharp"; bool isValid = name.Length > 5; int length = isValid;

d) Bool isEven = true; int num = 10; string result = isEven == (num % 2 == 0) ? "Par" : "Ímpar";

e) Long bigNumber = 9223372036854775807; bool isLarge = bigNumber > int.MaxValue; string description = "Número grande: " + isLarge;

Programação C# – String

- A classe String em C# representa uma sequência de caracteres e, em um nível mais técnico, uma instância da classe String é essencialmente um *array* de caracteres (char), mas a classe oferece uma ampla variedade de métodos que facilitam a manipulação de texto.

Alguns métodos:

- Clone(): cria uma cópia dessa instância.
- Compare(): compara duas *strings* e retorna um valor indicando se uma é menor, igual ou maior que a outra.
- Contains(): verifica se a *string* contém uma substring específica.
  - EndsWith(): determina se a *string* termina com a substring.
  - Replace(): substitui todas as ocorrências de uma substring especificada por outra substring.
  - Split(): divide a *string* em substrings com base em caracteres de delimitação.
  - Substring(): retorna uma substring de uma *string.*
  - ToLower() e ToUpper(): converte a string para minúsculas ou maiúsculas, respectivamente.

Programação C# – StringBuilder

- Pode ser usado quando modificações frequentes são necessárias.
- É uma classe em C# encontrada no namespace System.Text e representa uma sequência de caracteres mutável, especialmente útil quando precisamos realizar operações de manipulação de *string* frequentes ou complexas.
- É capaz de indicar quanto espaço de memória reservou (o que, aliás, é diferente do comprimento, que indica quantos caracteres estão atualmente na sequência).
- A capacidade pode aumentar automaticamente à medida que os caracteres são adicionados, mas também podemos definir a capacidade manualmente para otimizações.

Alguns métodos:

- Append: adiciona uma *string* ao final da sequência atual.
- Insert: insere uma *string* em uma posição específica.
- Remove: remove caracteres em um intervalo específico.
- Replace: substitui todas as ocorrências de uma substring especificada.
- Clear: limpa o conteúdo do StringBuilder.

Programação C# – Variáveis

- Permitem que os desenvolvedores armazenem e manipulem dados em seus programas.
- É um nome associado a uma localização de memória.
- Declaramos uma variável para reservar espaço na memória, e esse espaço é usado para armazenar valores, que podem ser alterados durante a execução do programa.
- O tamanho e o intervalo de valores que podem ser armazenados em variáveis numéricas dependem do tipo específico.
- Uma característica interessante do C# é sua capacidade de inferir tipos de variáveis. Utilizando a palavra-chave var, o compilador determina automaticamente o tipo de variável com base no valor atribuído.

Programação C# – Operadores

- São símbolos que realizam operações em variáveis e valores.
- C# contém um rico conjunto de operadores que permitem manipular variáveis de diversos tipos.
- São semelhantes a funções matemáticas e lógicas que se aplicam em operações básicas.

Operador condicional (?:):

- a?b:c
- retorna b se a for verdadeiro, e c caso contrário

Tipo de operador	Símbolo	Exemplo	Descrição
Aritmético	+	a + b	Adição
	-	a - b	Subtração
	*	a * b	Multiplicação
	/	a / b	Divisão
	%	a % b	Módulo (resto da divisão)
Comparação	>	a > b	Maior que
	<	a < b	Menor que
	>=	a >= b	Maior ou igual
	<=	a <= b	Menor ou igual
	==	a == b	Igualdade
	!=	a != b	Diferente
Lógico	&&	a && b	E lógico
	||	a || b	Ou lógico
	!	!a	Negação lógica
Atribuição	=	a = b	Atribui o valor de b a a
	+=	a += b	Atribui a soma de a e b a a
	-=	a -= b	Atribui a diferença entre a e b a a
	*=	a *= b	Atribui o produto de a e b a a

Fonte: autoria própria.

Programação C# – Expressões

- São combinações de operandos (como literais, variáveis, propriedades etc.) e operadores que, quando avaliados, retornam um único valor.
- São fundamentais para qualquer linguagem de programação, pois são a base da lógica e do cálculo C#, contém um rico conjunto de operadores que permitem manipular variáveis de diversos tipos.
- Podem ser tão simples quanto um valor literal ou tão complexas quanto uma combinação de várias subexpressões, funções e propriedades, sendo avaliadas para produzir um valor que pode ser usado, por exemplo, para tomar uma decisão, realizar um cálculo ou simplesmente exibir informações.

Não se limitam a cálculos aritméticos ou lógicos. Elas podem ser usadas em muitos contextos, como:

- atribuições: int totalRaios = raiosLancados;
- condições: if (eficaciaEncanto > 100) {...}
- retornos de método: return totalRaios;
- argumentos de método: LancarRaios(raiosLancados)

Programação C# – Declarações

- São instruções fundamentais que especificam como os dados são representados e manipulados no código. Elas incluem declarações de variáveis, constantes, condicionais, *loops*, métodos, classes e *namespaces.*
- A declaração *if* avalia uma expressão entre parênteses; se essa expressão resultar em verdadeiro (*true*), o bloco de código subsequente é executado. Caso contrário, se houver uma cláusula *else* após o *if*, o bloco de código associado ao *else* é executado.
- Um *loop* do tipo *for* repete um bloco de código por um número específico de vezes e é estruturado em três partes: inicialização da variável de controle; condição a ser avaliada para término das repetições; e iteração que representa a forma de atualização da variável de controle após execução de cada repetição.
  - Já o *loop* do tipo *while* repete um bloco de código enquanto uma condição for verdadeira.
  - A declaração *switch* permite selecionar um entre vários blocos de código para ser executado. É frequentemente usada como alternativa para longas cadeias de *if-else if-else.*

Programação C# – Declarações

```
1.  using System;
2.  class Program
3.  {
4.          static void Main(string[] args)
5.  {
6.          string deus = "Zeus";
7.
8.          // Declarações de Loop: For
9.          for (int i = 0; i  < 12; i ++)
10.         {
11.                 Console.WriteLine($"O {i + 1}° deus olímpico.");
12.         }
13.
14.         // Declarações de Loop: While
15.         int mortaisConvertidos = 0;
16.         while (mortaisConvertidos < 100)
17.         {
18.         // Zeus converte mais mortais
19.         mortaisConvertidos++;
20.         {
21.                 Console.WriteLine($" {mortaisConvertidos} mortais foram convertidos por Zeus.");
22.                 Console.ReadLine();
23.         }
24. }
```

Fonte: autoria própria.

Programação C# – Vetores

- Também conhecidos como *arrays* permitem armazenar vários itens do mesmo tipo em uma única variável, sendo usados extensivamente para armazenar, manipular e recuperar dados de forma eficiente.
- É uma coleção de variáveis do mesmo tipo, dispostas em sequência na memória e acessíveis por um índice.
- A declaração de um vetor envolve a especificação do tipo de seus elementos, seguido por colchetes, e então o nome do vetor.
- Para inicializar um vetor, o desenvolvedor pode definir explicitamente seu tamanho e/ou os valores que ele contém.
    - Vetores são baseados em zero, ou seja, o primeiro elemento tem índice 0, o segundo tem índice 1 e assim por diante.
    - Um elemento do vetor é acessado através de seu índice.
    - Um vetor multidimensional pode ser visualizado como matriz ou até mesmo como cubo de dados.

Programação C# – Vetores

```
5.  class Program
6.  {
7.  static void Main (string[ ] args)
8.  {
9.      // Declaração de um vetor com espaço para 5 inteiros.
10.     int [ ] numeros = new int [5];
11.
12.     // Declaração e inicialização de um vetor com nomes de deuses gregos.
13.     string[ ] deusesGregos = { "Zeus", "Hera", "Poseidon", "Demeter", "Ares");
14.
15.     // Acesso ao primeiro deus grego (baseado em zero, então [0] é "Zeus").
16.     string deus = deusesGregos [0];
17.     Console.WriteLine ($"O primeiro deus é: {deus}");
18.
19.     // Iterando através de todos os deuses e imprimindo cada um.
20.     for (int i = 0; i < deusesGregos.Length; i++)
21.     {
22.     Console.WriteLine($"Deus # {i + 1]: {deusesGregos[ i ]}");
23.     }
```

Fonte: autoria própria.

Programação C# – Array

- Os vetores em C# são objetos e, portanto, contêm métodos associados a eles. Por exemplo, o método Array.Resize pode ser usado para alterar o tamanho de um vetor existente.

- A classe Array fornece uma série de métodos estáticos úteis para manipular vetores.

Método	Descrição
Clear	Define um intervalo de elementos no array como o valor-padrão do tipo do array (normalmente zero)
Copy	Copia elementos de um array para outro
CopyTo	Copia todos os elementos do array unidimensional atual para um array unidimensional compatível
Exists	Determina se um elemento satisfaz às condições definidas por um predicado especificado
Find	Procura um elemento que corresponda às condições definidas por um predicado especificado
FindAll	Obtém todos os elementos que correspondem às condições definidas por um predicado especificado
FindIndex	Retorna o índice do primeiro elemento encontrado que corresponde às condições
FindLast	Retorna o último elemento encontrado que corresponde às condições
FindLastIndex	Retorna o índice do último elemento encontrado que corresponde às condições
ForEach	Executa a ação especificada em cada elemento de um array
IndexOf	Retorna o índice da primeira ocorrência de um valor em um array unidimensional
LastIndexOf	Retorna o índice da última ocorrência de um valor em um array unidimensional
Reverse	Inverte a ordem dos elementos em um array unidimensional inteiro ou em uma porção do array
Sort	Classifica os elementos em um array inteiro ou em uma porção de um array unidimensional

Fonte: autoria própria.

Programação C# – Enumeração

- É um tipo distinto, que consiste em um conjunto de constantes nomeadas, essencialmente grupos de constantes inteiras com nomes que facilitam a leitura e manutenção de um programa.
- Por padrão, o primeiro membro de um enum tem valor 0, e o valor de cada membro subsequente é incrementado em 1. É possível, entretanto, especificar valores para os membros, se necessário.
- São úteis porque melhoram a legibilidade do código e reduzem a possibilidade de erro ao restringir valores a um conjunto finito de opções.

```
enum DiaDaSemana
{
    Domingo = 0,
    Segunda = 1,
    Terça = 2,
    Quarta = 3,
    Quinta = 4,
    Sexta = 5,
    Sábado = 6
}
```

Fonte: autoria própria.

Programação C# – Estruturas

- Estrutura (ou *struct*) é um tipo de valor que pode ser usado para encapsular pequenos grupos de variáveis relacionadas.
- Uma das principais diferenças entre uma *struct* e uma classe (*class*) é que estruturas são tipos de valor, enquanto classes são tipos de referência, ou seja, quando uma estrutura é passada para uma função, ela é passada por valor (ou seja, uma cópia dos dados é feita), enquanto uma classe é passada por referência.
- Estruturas e classes têm propósitos diferentes, e escolher entre elas depende das necessidades do problema que desejamos resolver.

Fonte: autoria própria.

Característica	Estruturas	Classes
Alocação	Pilha (*stack*)	Heap
Coleta de lixo	Não está sujeita	Está sujeita
Herança	Suporta apenas herança de interfaces	Suporta herança completa
Construtor-padrão	Tem construtor-padrão não modificável	Pode ou não ter
Imutabilidade	Frequentemente projetada para ser imutável	Pode ser mutável ou imutável
Semântica de passagem	Passada por valor (cópia)	Passada por referência
Eficiência em dados (pequenos)	Geralmente mais eficiente em pequenos volumes de dados	Potencialmente menos eficiente
Eficiência em dados (grandes)	Pode ser menos eficiente (cópia de dados)	Mais eficiente (passagem de referência) para grandes volumes de dados
Uso recomendado	Tipos leves que representam um único valor	Tipos mais complexos ou que necessitam de herança, polimorfismo etc.

Interatividade

Qual das seguintes afirmações está correta?

a) Em C#, as estruturas são tipos de referência usados para armazenar dados relacionados.

b) Enumerações são usadas para definir um conjunto de constantes nomeadas e são mutáveis.

c) As variáveis em C# devem ser inicializadas no momento da declaração.

d) Operadores em C# são usados para executar operações em variáveis e valores, e expressões são combinações de operadores e operandos que produzem um resultado.

e) Declarações em C# são instruções que não retornam um valor e não podem ser usadas em expressões.

Resposta

Qual das seguintes afirmações está correta?

a) Em C#, as estruturas são tipos de referência usados para armazenar dados relacionados.

b) Enumerações são usadas para definir um conjunto de constantes nomeadas e são mutáveis.

c) As variáveis em C# devem ser inicializadas no momento da declaração.

d) Operadores em C# são usados para executar operações em variáveis e valores, e expressões são combinações de operadores e operandos que produzem um resultado.

e) Declarações em C# são instruções que não retornam um valor e não podem ser usadas em expressões.

Programação C# – Object

- É o tipo raiz na hierarquia de tipos, ou seja, todas as classes, independentemente de sua posição ou complexidade na hierarquia de herança, derivam direta ou indiretamente do tipo Object.

- Esse tipo raiz é definido no *namespace* System.

O tipo Object contém um conjunto mínimo de métodos que todos os objetos em C# herdam e podem sobrescrever, incluindo funções como:

- ToString: retorna uma representação de *string* do objeto.
- GetHashCode: obtém um código *hash* para o objeto atual.
- Equals: determina se um objeto é igual a outro.

```
// Armazenando entidades em variáveis do tipo Object
Object obj1 = zeus;
Object obj2 = medusa;
```

Fonte: autoria própria.

Programação C# – Entrada e saída (E/S)

- Muitas vezes referidas como I/O (do inglês *input/output*), são uma parte fundamental de qualquer linguagem de programação, pois determinam como os programas interagem com fontes de dados externas, como arquivos, bancos de dados, redes ou entrada-padrão/saída-padrão de um sistema.
- São tratadas principalmente através da Biblioteca de Classes Base (BCL) do .NET Framework. Essa biblioteca fornece uma ampla gama de classes e métodos que permitem aos desenvolvedores ler e escrever dados de e para várias fontes.
- Há dois grandes domínios de entrada e saída: I/O de arquivos e I/O de *streams.*
    - A operação de arquivos é facilitada por uma série de classes que permitem ler e escrever em arquivos no sistema.
    - Essas operações podem ser tão simples quanto ler ou escrever todo o conteúdo de um arquivo de texto, ou tão complexas quanto manipular arquivos binários ou serializar e desserializar objetos.

Programação C# – Arquivos (E/S)

- Para diversas operações, arquivo é uma entidade que pode ser aberta, lida, escrita e fechada, por isso cuidados devem ser tomados para tratar exceções.
- Os dois principais tipos de arquivo são textuais e binários.
- O acesso ao sistema de arquivos é, por natureza, mais lento que muitas outras operações em um programa, como fazer cálculos ou acessar a memória.

O *namespace* System.IO contém as classes primárias para operações de arquivo:

- File: oferece métodos estáticos para operações de arquivo, incluindo ler e escrever em arquivos, verificar a existência de um arquivo, excluí-lo, copiá-lo e muitas outras operações.
  - FileInfo: representa um arquivo no sistema de arquivos. Oferece propriedades para obter informações sobre o arquivo, como seu caminho, tamanho e datas de criação e modificação. Além disso, fornece métodos de instância para criação, exclusão, movimentação e abertura de arquivos.
  - Directory e DirectoryInfo: essas classes são para operações relacionadas a diretórios.

Programação C# – Streams (E/S)

- Uma *stream* é uma abstração de uma sequência de bytes. Pode-se pensar nela como um fluxo contínuo de dados que pode ser lido ou escrito.
- Essa abstração é poderosa porque permite que os desenvolvedores tratem diversas fontes de dados – sejam arquivos, redes, memória etc. – de maneira consistente.
- A BCL (Base Class Library) fornece diversas classes para trabalhar com *streams*, e estão principalmente no *namespace* System.IO.
- Através dessas classes, podemos ler ou escrever em fluxos de dados, bem como encadear múltiplos fluxos juntos para realizar operações complexas, como comprimir dados ou criptografia.

  - A principal classe que representa um fluxo contínuo de dados em .NET é a classe Stream, da qual todas as outras classes de *stream* derivam.
  - Essa classe abstrata define métodos básicos, como Read, Write, Flush, Seek, entre outros, fundamentais para manipular dados.
  - Em sua essência, *streams* leem e escrevem dados.

Programação C# – Streams (E/S)

Existem três tipos básicos de manipulação de fluxos contínuos de dados:

- MemoryStream representa um *stream* que utiliza a memória como armazenamento, sendo útil quando precisamos armazenar temporariamente dados em memória antes de movê-los para um destino diferente.
- NetworkStream é usado para ler e escrever em conexões de rede, comumente usado em classes como TcpClient e TcpListener para comunicações baseadas TCP.
- BufferedStream dá suporte para *buffering*, o que pode melhorar a performance de operações de I/O ao ler ou escrever grandes quantidades de dados em blocos em vez de byte a byte.
    - O método Flush garante que todos os dados em um buffer sejam escritos em um fluxo de dados subjacente.
    - Também é importante fechar ou descartar um *stream* quando ele não é mais necessário, liberando recursos associados.

Programação C# – Assíncrona (E/S)

- Se refere a operações de entrada/saída (I/O) realizadas sem bloquear a execução do programa.
- Dada a natureza potencialmente demorada das operações de E/S, o C# oferece mecanismos para essas operações de forma assíncrona, ou seja, podemos iniciar uma operação de E/S e, em vez de esperar que ela termine, seu programa pode executar outras tarefas.
- Quando a operação de I/O estiver concluída, seu programa será notificado e poderá processar os resultados.
- Essas operações são comuns quando trabalhamos com arquivos, redes ou qualquer outra operação que possa levar tempo significativo e não queremos bloquear a *thread* principal.
    - O .NET Framework e o .NET Core (e o .NET 5 e versões posteriores) fornecem *application programming interfaces* (APIs) para I/O assíncrona.
    - A partir do C# 5, com a introdução das palavras-chave *async* e *await*, tornou-se ainda mais fácil escrever código-fonte de forma limpa e legível.

Programação C# – Além do System.IO (E/S)

Além do *namespace* System.IO, diversas outras partes do *framework* .NET fornecem mecanismos para interação de I/O:

- o *namespace* System.Net.Sockets é essencial para comunicações de rede. Dentro dele encontramos classes que permitem a comunicação utilizando *sockets*.
- a classe WebClient, que está sob o domínio do *namespace* System. Net. Ela é projetada especificamente para enviar e recuperar informações de recursos da *web*, simplificando a comunicação com serviços digitais.
- o *namespace* System.Data. SqlClient abriga classes como SqlConnection e SqlCommand, que juntas possibilitam uma ampla gama de operações de I/O com bancos de dados.
    - em interfaces gráficas para *desktop*, a classe System.Windows.Forms.TextBox é um exemplo de componente que facilita a entrada e a saída de texto.
    - a classe System.Console, amplamente utilizada para operações básicas de E/S em aplicativos de console. Essa classe proporciona uma interação direta com o usuário por texto, permitindo ler e escrever na janela do console.

C# e .NET

- C# é uma linguagem de programação moderna (Albahari, 2021), orientada a objetos, desenvolvida pela Microsoft como parte de sua iniciativa .NET e projetada para ser uma plataforma multilinguagem (Troelsen; Japikse, 2021).
- Várias linguagens de programação podem ser usadas para desenvolver aplicações para a plataforma .NET, dentre as quais: Visual Basic .NET (VB.NET), F#, C++/CLI, IronPython, IronRuby, ASP.NET.
- A plataforma .NET, por sua vez, é um *framework* (ou arcabouço) de desenvolvimento que proporciona um vasto conjunto de bibliotecas e ferramentas para auxiliar os desenvolvedores a criar e executar aplicações para Windows, *web*, *mobile*, entre outros (Freeman, 2019).
    - C# foi projetada para ser a linguagem principal da plataforma .NET.
    - É importante notar que o termo <u>biblioteca</u> é uma coleção de funções, classes e métodos que podem ser utilizados por outros programas. Já um <u>*framework*</u> fornece uma estrutura e conjunto de regras para desenvolver aplicações, estabelecendo uma abordagem ou arquitetura específica.

CLR – *Common Language Runtime*

- Um código escrito em C# é compilado para uma linguagem intermediária chamada Common Intermediate Language (CIL).
- CIL é uma abstração interpretada pela máquina virtual .NET (conhecida como CLR, ou Common Language Runtime).
- Compilado o código C# para CIL, ele pode ser executado em qualquer sistema em que o CLR esteja presente, garantindo a portabilidade do .NET.
- O CLR atua como essa máquina virtual e é responsável por carregar e executar o código CIL (Richter, 2012).

  - Ao executarmos um programa .NET, ele compila *just in time* (JIT) o CIL para o código de máquina específico da arquitetura em que o programa está sendo executado (processo feito em tempo de execução).
  - O compilador JIT traduz o CIL em instruções que o sistema operacional e o *hardware* do computador podem entender – ou seja, o mesmo código binário .NET pode ser executado em diferentes sistemas operacionais e *hardwares.*

CLR – A base sobre a qual os programas .NET são executados

- Além de gerenciar a execução do código, o CLR fornece vários outros serviços importantes, como o gerenciamento de memória através do coletor de lixo (*Garbage Collector*).
- Oferece suporte a funcionalidades relacionadas à segurança, como a verificação de tipos e a execução de código em ambientes restritos (*sandboxing*).
- Fornece serviços para facilitar a interoperabilidade entre códigos gerenciados (.NET) e não gerenciados (como APIs escritas em C ou C++).
- A plataforma .NET inclui uma biblioteca extensa de classes (Framework Class Library ou FCL) que funciona em conjunto com o CLR para fornecer uma ampla variedade de funcionalidades, desde operações básicas de entrada e saída até serviços *web* avançados.

BCL – *Base Class Library*

Classe/namespace	Descrição
System.Object	Base para todas as classes, fornece métodos fundamentais
System.String	Representa e manipula sequências de caracteres
System.Array	Classe-base para todos os vetores
System.Int32, System.Double, System. Boolean	Representam tipos primitivos e suas operações
System.Collections.Generic.List<T>	Estrutura de dados para lista de objetos
System.Collections.Generic. Dictionary<TKey, TValue>	Estrutura de dados para mapear chaves a valores
System.IO.File & System.IO.Directory	Manipulam arquivos e diretórios
System.IO.Stream & System. IO.FileStream & System.IO.TextWriter	Classes relacionadas à entrada/saída de dados, como leitura e escrita em arquivos
System.Net.Http.HttpClient	Usado para enviar e receber solicitações HTTP
System.Linq.Enumerable	Fornece capacidades de consulta para coleções com LINQ
System.Threading.Tasks.Task	Representa operações assíncronas

Fonte: autoria própria.

- A Base Class Library (BCL) é uma componente vital da plataforma .NET, da qual o C# é uma das linguagens principais.
- É uma coleção extensiva de classes, interfaces e estruturas que fornecem funcionalidades predefinidas para tarefas comuns de programação.
- É um conjunto de ferramentas e utilitários que os desenvolvedores podem aproveitar sem precisar “reinventar a roda” para tarefas básicas, como manipulação de *strings*, acesso a arquivos, operações de rede, coleções e muitos outros.
- A BCL e o CLR são os dois pilares que sustentam a experiência de desenvolvimento no .NET.

Interatividade

Qual das seguintes alternativas é correta?

a) A classe FileStream é usada para manipulação de memória, não de arquivos.
b) C# suporta apenas operações de I/O síncronas, sem suporte para operações assíncronas.
c) O CLR é responsável pela compilação Just-In-Time (JIT), gerenciamento de memória e segurança de tipo, enquanto o BCL fornece uma coleção extensiva de classes predefinidas.
d) A leitura e escrita de arquivos em C# são realizadas automaticamente, sem a necessidade de fechar explicitamente os arquivos.
e) O BCL fornece funcionalidades para interfaces gráficas, mas não inclui classes para operações de I/O ou manipulação de *strings.*

Resposta

Qual das seguintes alternativas é correta?

a) A classe FileStream é usada para manipulação de memória, não de arquivos.

b) C# suporta apenas operações de I/O síncronas, sem suporte para operações assíncronas.

c) O CLR é responsável pela compilação Just-In-Time (JIT), gerenciamento de memória e segurança de tipo, enquanto o BCL fornece uma coleção extensiva de classes predefinidas.

d) A leitura e escrita de arquivos em C# são realizadas automaticamente, sem a necessidade de fechar explicitamente os arquivos.

e) O BCL fornece funcionalidades para interfaces gráficas, mas não inclui classes para operações de I/O ou manipulação de *strings.*

Arquitetura – 3 camadas

- No desenvolvimento de *software*, a forma como estruturamos e organizamos o código não é apenas uma questão de estética ou preferência, mas uma decisão que tem implicações diretas na escalabilidade, modularidade, manutenção e desempenho das aplicações.
- Uma solução é a estruturação em uma arquitetura em três camadas distintas: apresentação, lógica de negócios e dados.

Fonte: autoria própria, com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Arquitetura – 3 camadas (aplicação)

- A camada de apresentação é a interface com a qual o usuário tem contato direto.
- Pode ser uma interface gráfica tradicional em um aplicativo *desktop*, uma página *web* em um navegador ou até mesmo os *endpoints* de uma API em serviços mais modernos.
- Ela é crucial, pois define como o usuário percebe e interage com a aplicação.

Uma boa camada de apresentação é intuitiva e responsiva às ações do usuário:

- em aplicações para *desktop* tradicionais, como as criadas usando Windows Forms ou Windows Presentation Foundation (WPF), C# é a linguagem principal;
    - para aplicações *web*, a camada de apresentação muitas vezes é construída usando HTML, CSS e JavaScript, no entanto com o uso de tecnologias como ASP.NET Core MVC ou Blazor, a lógica do lado do servidor dessa camada é escrita em C#;
    - em aplicações móveis usando Xamarin, C# é a linguagem principal para criar a interface e a lógica do aplicativo.

Arquitetura – 3 camadas (lógica de negócios e dados)

- O coração de muitas aplicações reside na camada de lógica de negócios.
- As regras são definidas, os cálculos são feitos e as operações centrais da aplicação acontecem.
- Funciona como uma mediadora: recebe informações da camada de apresentação, processa-as conforme as regras estabelecidas e interage com a camada de dados, quando necessário.
- Independentemente da plataforma ou do tipo de aplicação, o C# é frequentemente usado para codificar a lógica de negócios, especialmente em soluções .NET.
    - A camada de dados é o repositório de informações da aplicação. Seja em bancos de dados relacionais tradicionais, sistemas de arquivos ou soluções mais contemporâneas baseadas em nuvem, essa camada lida com a persistência, recuperação e atualização dos dados.

Arquitetura – 3 camadas (lógica de negócios e dados)

Fontes: autoria própria, com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Arquitetura – 3 camadas na prática

Estrutura:

1. Camada de apresentação (usando WPF)	
DeusView.xaml	Define a interface gráfica para visualizar detalhes de um deus grego
DeusView.xaml.cs	É o código-behind que interage com a interface gráfica
2. Camada de lógica de negócios (domínio)	
ServicoDeDeus.cs	Contém a classe ServicoDeDeus, que define os métodos para buscar e manipular informações relacionadas aos deuses gregos
DeusGrego.cs	Contém a definição do modelo (classe) DeusGrego, que representa um deus grego com propriedades como nome, domínio e símbolo
3. Camada de acesso a dados	
IRepositorioDeDeus.cs	Define a interface IRepositorioDeDeus, que especifica os métodos que um repositório de deuses deve implementar
RepositorioDeDeus.cs	Implementação concreta da interface IRepositorio De Deus, que contém a lógica para buscar um deus na fonte de dados

Fonte: autoria própria.

Arquitetura – 3 camadas na prática: apresentação

- Apresentação
- DeusView.xaml

```
1.    <Window x:Class="MitologiaGrega. DeusView"
2.            xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.            xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.            Title="Detalhes do Deus Grego" Height="200" Width="300">
5.            <Grid>
6.                    <Label Content="Nome do Deus:" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,10,0,0"/>
7.                    <TextBox Name="nomeTextBox" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,30,10,0" Height="25" Width="260"/>
8.                    <Button Content="Buscar" HorizontalAlignment="Right"
VerticalAlignment="Top" Margin="0,60,10,0" Width="80" Click="BuscarDeus"/>
9.                    <TextBlock Name="detalhesTextBlock" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,100,10,0"/>
10.           </Grid>
11.   </Window>
```

Fonte: autoria própria.

Arquitetura – 3 camadas na prática: apresentação

- Apresentação
- DeusView.xaml.cs
- Código-behind

```
1.  public partial class DeusView : Window
2.  {
3.      private ServicoDeDeus  _servicoDeDeus;
4.
5.      public DeusView()
6.      {
7.          InitializeComponent();
8.          _servicoDeDeus = new ServicoDeDeus (new RepositorioDeDeus());
9.      }
10.     private void BuscarDeus (object sender, RoutedEventArgs e)
11.     {
12.         var deus = _servicoDeDeus. BuscarDeusPorNome (nomeTextBox.Text);
13.         if  (deus ! = null)
14.         {
15.             detalhes TextBlock.Text = $"Nome: {deus. Nome)\nDomínio:
{deus. Dominio}\nSímbolo: (deus.Simbolo}";
16.         }
17.         else
18.         {
19.             detalhesTextBlock.Text = "Deus não encontrado.";
20.         }
21.     }
22. }
```

Fonte: autoria própria.

Arquitetura – Model-View-Controller (MVC)

É amplamente reconhecido (Reenskaug, 2003) e segmenta a aplicação em três pilares:

- *Model:* abriga os dados e a lógica de negócios.
- *View:* foca a apresentação e a interação com o usuário.
- *Controller:* age como orquestrador entre o *Model* e a *View*.

Fonte: autoria própria, com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

- É intrinsecamente associado ao *framework* ASP.NET Core MVC da Microsoft, uma das principais ferramentas para desenvolvimento web no universo .NET.
- Esse padrão de design tem suas origens no final dos anos 1970 e não há uma correlação exata entre os componentes do MVC e a arquitetura de três camadas.

Arquitetura – Model-View-ViewModel (MVVM)

- É uma escolha muito usada para aplicações WPF e Xamarin, voltadas respectivamente para os ambientes *desktop* e *mobile*.
- Foi popularizado por John Gossman em 2005, quando trabalhava na Microsoft. Ele introduziu o MVVM como uma adaptação especializada do padrão Presentation Model para ser usado com o WPF, a nova tecnologia de UI da Microsoft na época.
- Foi desenvolvido com a ideia de aproveitar ao máximo os recursos de vinculação de dados (*data binding*) oferecidos pelo WPF, permitindo uma separação mais clara da lógica de apresentação dos componentes da interface do usuário.
  - Ganhou popularidade não apenas em aplicações WPF, mas também em outras tecnologias e plataformas, incluindo desenvolvimento móvel e aplicações *web* de página única.

Fonte: autoria própria, com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Arquitetura – Diferentes modelos

- A escolha da arquitetura depende do contexto.
- Todas são usadas.
- Pode ter mais de uma.

3 camadas

- Quando a aplicação tem forte componente de processamento de negócios separável das demais camadas
- Quando a aplicação precisa ser escalável horizontalmente
- Quando é necessário isolar a lógica de negócios para reutilização e manutenção
- Sistemas de ERP, sistemas de CRM e outras aplicações empresariais complexas

Model-View-Controller (MVC)

- Principalmente para aplicações web
- Quando se busca uma clara separação entre lógica de apresentação, dados e controle de fluxo
- Quando a testabilidade é importante
- Foco em ações orientadas a controladores (manipulação de solicitações HTTP)
- Aplicações web baseadas em ASP.NET MVC, Ruby on Rails, Spring MVC

Model-View-ViewModel (MVVM)

- Principalmente para *rich client applications*, como aplicações desktop ou móveis
- Quando você está usando frameworks com vinculação de dados bidirecional (*data binding*)
- Quando a lógica da interface do usuário é complexa e precisa ser separada da visualização
- Aplicações desenvolvidas em WPF, Xamarin.Forms, SPA (Angular, Vue.js)

Conversões

Existem dois tipos principais de conversão em C#:

- Implícitas: ocorrem automaticamente quando se passa o valor de um tipo menor (que ocupa menos espaço em bytes na memória) para um tipo maior (que requer mais espaço em memória); por exemplo, é seguro converter um int num double sem perder informação.
- Explícitas: frequentemente referidas como *casting*, são necessárias quando há risco de perder dados; por exemplo, ao converter um double num int, a parte fracional do double será descartada. Em tais casos, o programador deve informar explicitamente o compilador de que está ciente da potencial perda de dados, utilizando a sintaxe de *casting.*

  - Tipos numéricos têm métodos como ToString, Parse e TryParse, vitais para converter números em *strings* e vice-versa.
  - A capacidade de trabalhar com números de forma eficiente e precisa é essencial para muitas aplicações, e o C#, reconhecendo essa importância, fornece ferramentas robustas, como: Convert, Numerics e Math.

Trabalhando com texto e datas

Trabalhar com texto envolve compreender as capacidades e particularidades da linguagem em relação à manipulação de *strings.* Existem três técnicas comuns:

- concatenação é o processo de unir duas ou mais *strings* para formar uma única. Uma das formas mais diretas é usar o operador +. Outra maneira é usar o método String.Concat;
- interpolação oferece uma maneira de inserir valores de variáveis ou expressões diretamente dentro de uma *string.* Pode ser realizada usando o símbolo $ antes da *string* e colocando as variáveis ou expressões entre chaves { };
- a formatação de texto refere-se ao processo de ajustar a aparência do texto ou estrutura. Neste caso, utiliza-se o método String.Format.

Trabalhando com texto e datas

Para lidar com datas, horas e intervalos de tempo temos o *namespace* System.DateTime:

- a estrutura DateTime, que serve para representar um instante específico, pode expressar tanto uma data quanto uma hora específica. Para UTC, temos o DateTimeOffset;
- o *TimeSpan* representa um intervalo de tempo e pode ser determinado subtraindo-se duas datas;
- a manipulação adequada de zonas horárias e até mesmo do horário de verão é viabilizada pela classe TimeZoneInfo.

Breve histórico da linguagem C# e do .NET

- Antes de lançar o C#, a Microsoft usava várias linguagens de desenvolvimento, sendo o Visual Basic (VB) e o Visual C++ as mais notáveis.
- A ascensão do Java, especialmente no espaço empresarial, estimulou a Microsoft a desenvolver sua própria versão do Java chamada J++ (Visual J++), porém disputas legais levaram a Microsoft a criar sua própria plataforma e linguagem.
- O .NET foi lançado oficialmente em 2002 e introduziu uma variedade de inovações (Box, 2002). A ideia era fornecer uma plataforma única para desenvolver aplicações *web*, *desktop*, móveis e de serviços, em contraste direto com a abordagem tradicional, que exigia diferentes ferramentas e linguagens para diferentes tipos de aplicações.
- A Microsoft lançou uma nova linguagem chamada C# (Hejlsberg; Torgensen, 2010).
    - Por volta de 2005, com o lançamento do .NET Framework 2.0 e do Visual Studio 2005, a Microsoft afirmou que havia mais de três milhões de desenvolvedores .NET em todo o mundo.
    - A evolução do .NET Core culminou no lançamento do .NET 5 em 2020, que unificou os vários sabores do .NET e posicionou a plataforma no futuro.

Interatividade

Tendo em mente os modelos de arquitetura, qual das seguintes alternativas é correta?

a) A arquitetura de 3 camadas consiste em Cliente, Servidor e Banco de Dados.
b) MVVM é um padrão de projeto mais antigo que MVC, sendo mais adequado para aplicações web.
c) No MVC, a View interage diretamente com o Model, e no MVVM o Controller atua como intermediário entre a View e o ViewModel.
d) MVC e MVVM não suportam o conceito de data binding.
e) No MVC, o Controller manipula a lógica de negócios, enquanto no MVVM o ViewModel facilita a comunicação entre a View e o Model, promovendo o binding de dados.

Resposta

Tendo em mente os modelos de arquitetura, qual das seguintes alternativas é correta?

a) A arquitetura de 3 camadas consiste em Cliente, Servidor e Banco de Dados.

b) MVVM é um padrão de projeto mais antigo que MVC, sendo mais adequado para aplicações web.

c) No MVC, a View interage diretamente com o Model, e no MVVM o Controller atua como intermediário entre a View e o ViewModel.

d) MVC e MVVM não suportam o conceito de data binding.

e) No MVC, o Controller manipula a lógica de negócios, enquanto no MVVM o ViewModel facilita a comunicação entre a View e o Model, promovendo o binding de dados.

Referências

- ALBAHARI, J. *C# 9.0 in a nutshell*: the definitive reference. Sebastopol: O'Reilly Media, 2021
- BOX, D. *Essential.NET*: the common language runtime. [S.l.]: Addison-Wesley, 2002.
- FREEMAN, A. *Pro ASP.NET Core 3*: develop cloud-ready web applications using MVC 3, blazor, and razor pages. [S.l.]: Apress, 2019.
- HEJLSBERG, A.; TORGENSEN, M. *The C# programming language*. [S.l.]: Addison-Wesley, 2010.
- REENSKAUG, T. *The model-view-controller (MVC), its past and present*. [S.l.]: Computer Science, 2003.
- RICHTER, J. *CLR via C#*. [S.l.]: Microsoft Press, 2012
- TROELSEN, A.; JAPIKSE, P. *Pro C# 9 with.NET 5*. New York: Apress Berkeley, 2021.

ATÉ A PRÓXIMA!